

Rafeeq Analytics

Backend Architecture & Data-Flow Reference

FastAPI · Google BigQuery · Gemini · scikit-learn / XGBoost
Customer-experience intelligence platform for a Qatar food-delivery operation

Technical documentation for engineers and external reviewers

Rafeeq Analytics — Backend

A single FastAPI service that turns raw call-centre transcripts, customer messages, and order data (all living in Google BigQuery) into classified, analysed, and predicted customer-experience intelligence, using Google Gemini for language tasks and a trained ML model for cancellation risk.

0. Table of Contents

- 1. System Overview & Tech Stack
- 2. Application Architecture
- 3. External Systems & Authentication
- 4. Configuration
- 5. Data Stores (BigQuery)
- 6. Cross-Cutting Services
- 7. Pillar 01 — Call Intelligence
- 8. Pillar 02 — Support Messages / Text Sentiment
- 9. Pillar 03 — Cancellation Prediction
- 10. Conversational Assistant (/chat)
- 11. Caching & Cost-Safety
- 12. Offline Scripts & Pipelines
- 13. Complete Endpoint Reference

1. System Overview & Tech Stack

The backend is one deployable FastAPI app exposing a REST API on **port 8000**. It is organised into three functional "pillars" plus a conversational assistant. All persistent data lives in Google BigQuery; the service holds no database of its own.

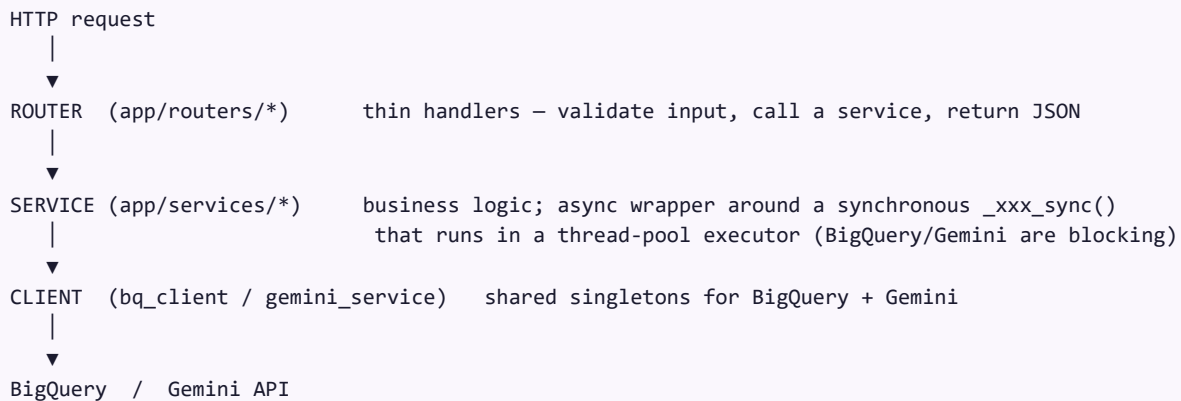
Concern	Technology
Web framework	FastAPI 0.115 + Uvicorn (ASGI)
Data warehouse	Google BigQuery (<code>google-cloud-bigquery</code>) — project <code>long-ceiling-343505</code> , dataset <code>reports</code>
LLM	Google Gemini (<code>google-generativeai</code>), model <code>gemini-3.5-flash</code>
ML (cancellation)	scikit-learn, XGBoost, LightGBM, Optuna (tuning), SHAP (explanations), imbalanced-learn
Validation	Pydantic v2 + pydantic-settings
Data wrangling	pandas, numpy
Packaging	Docker (python:3.12-slim), <code>uvicorn app.main:app</code>

Legacy files: the repository root contains an older single-file implementation (`main.py` , `analytics.py` , `bigquery_client.py` , `gemini_client.py`). These are **superseded** by the modular `app/` package. The

Dockerfile and README both run `app.main:app`; the root files are not part of the live service and can be ignored when reading the system.

2. Application Architecture

The app follows a clean three-layer convention everywhere:



Startup (lifespan)

`app/main.py` wires everything together. On startup it bootstraps the BigQuery tables it owns:

```
async def lifespan(app):
    await ensure_call_table()      # Pillar 01: reports.call_analysis
    await bootstrap_text_tables() # Pillar 02: messages / classifications / labels
    yield
```

Routers are registered in `app/main.py`. Pillar 03 (cancellation) is imported *defensively* inside a `try/except` — if the heavy ML libraries are not installed, that feature is simply disabled and the rest of the API still boots.

Router map

Router file	Prefix	Pillar
health.py	/health , /api/v1/health	shared
calls.py	/analyse , /predict	01
call_analytics.py	/calls , /analytics/*	01
messages.py	/api/v1/messages/*	02
sentiment.py	/api/v1/sentiment/*	02
text_analytics.py	/api/v1/analytics/*	02
chat_analytics.py	/api/messages/*	02 (chat history)
chat.py	/chat	assistant
cancellation.py	/api/cancellation/*	03

3. External Systems & Authentication

System	How it authenticates	Used for
BigQuery	Application Default Credentials (ADC) — <code>gcloud auth application-default login</code> , or a mounted service-account on Docker	All reads/writes. One shared <code>bigquery.Client</code> singleton.
Gemini	<code>GEMINI_API_KEY</code> from <code>.env</code>	Transcript/message classification, cancellation explanations, conversational chat, drivers report.

The API itself is **unauthenticated** — it trusts the frontend / network boundary. CORS is restricted to the configured origins (localhost:3000/3001 by default).

4. Configuration

`app/config.py` defines a Pydantic `Settings` model loaded from `.env` , cached with `@lru_cache` so it is read once per process.

Setting	Default	Purpose
<code>gemini_api_key</code>	(required)	Gemini auth
<code>gemini_model</code>	<code>gemini-3.5-flash</code>	Model id for all LLM calls
<code>gemini_concurrency</code>	5	Max simultaneous Gemini calls (semaphore)
<code>gcp_project_id</code>	<code>long-ceiling-343505</code>	BigQuery project
<code>bq_calls_dataset</code> / <code>bq_text_dataset</code>	<code>reports</code>	Datasets for Pillar 01/03 and Pillar 02
<code>bq_max_bytes_billed</code>	50 GiB	Per-query scan cap — BigQuery rejects any unbounded query above this
<code>cors_origins</code>	localhost:3000,3001	Allowed browser origins
<code>classify_batch_size</code> / <code>classify_batch_delay_s</code>	10 / 0.5s	Pillar 02 batch classification pacing

5. Data Stores (BigQuery)

Everything lives in the `reports` dataset of project `long-ceiling-343505`.

Table	Owner	Written by	Contents
<code>call_analysis</code>	Pillar 01	App (on analyse)	Gemini-analysed transcripts: intents, sentiment, entities (JSON), summary
<code>vendor_kpi</code>	read-only source	External pipeline	Raw per-order operational data — the backbone for Pillar 03 and order enrichment
<code>vendor_items_kpi</code>	read-only source	External	Per-order line items
<code>messages</code>	Pillar 02	App (ingest) + backfill scripts	Inbound customer messages (PII-redacted), <code>closed_at</code> , <code>agent_name</code>
<code>classifications</code>	Pillar 02	App (classify)	Gemini sentiment/intent/negative-trigger per message
<code>labels</code>	Pillar 02	Manual / seed	Human ground-truth for accuracy evaluation
<code>chat_history</code>	source	External	Raw chat logs — backfill source for message enrichment / SLA
<code>cancellation_predictions</code>	Pillar 03	App (lazily)	Persisted predictions — a read-through cache so orders aren't re-scored/re-billed

Tables the app owns are created on demand (`CREATE TABLE IF NOT EXISTS` / `create_table` catching `Conflict`), so a fresh environment self-bootstraps.

6. Cross-Cutting Services

Service	Responsibility
<code>bq_client.py</code>	Lazily builds one <code>bigquery.Client</code> with a default <code>maximum_bytes_billed</code> job config (cost safety). Shared by every service.
<code>gemini_service.py</code>	One <code>GenerativeModel</code> (JSON mode, temp 0.1) + a concurrency semaphore. <code>call_with_retry()</code> runs the blocking call in a thread, retries once, strips markdown fences.
<code>chat_service.py</code>	A <i>separate</i> Gemini model (temp 0.7, natural text) for the conversational assistant; injects pre-queried analytics as grounded context with a strict system prompt.
<code>helpers.py</code>	PII redaction (email/phone), Unicode cleaning, tolerant LLM-JSON extraction (<code>extract_json</code> with balanced-brace + trailing-comma salvage), agent-name / channel derivation.

7. Pillar 01 — Call Intelligence

Turns call-centre transcripts (English / Arabic / mixed) into structured analysis and dashboard aggregates.

7.1 Analysis flow (write path)

```
POST /analyse or /analyse/batch (≤100 transcripts)
|
▼ calls.py::_process(transcript)
1. call_service.analyse_transcript() → Gemini (JSON prompt)
   returns {intents[], sentiment, sentiment_confidence, entities{order_ids,
   restaurant_names, areas, product_names, qar_amounts}, summary}
2. if entities.order_ids: bq_calls.enrich_orders(order_ids)
   → BigQuery vendor_kpi + vendor_items_kpi (parametrised IN UNNEST)
3. asyncio.create_task( bq_calls.save_call_analysis(result) ) # fire-and-forget insert
4. clear_cache() # drop stale analytics so new calls show immediately
|
▼ returns the full analysis (with enriched_orders) to the caller
```

Concurrency is capped by a semaphore (5). `/predict` is a lighter variant: it classifies but skips BigQuery enrichment and persistence — suitable for real-time scoring.

The classification prompt (`call_service.py`)

A fixed instruction block forces English intent/sentiment labels regardless of transcript language, extracts order IDs in any format (incl. Arabic numerals), and demands strict JSON. The response is `_normalise()` d: intents filtered to a valid set (fallback `general_inquiry`), sentiment to {positive, neutral, negative}, confidence clamped 0–1.

7.2 Analytics flow (read path)

`call_analytics_service.py` runs aggregate SQL over `call_analysis` (joined to `vendor_kpi` via extracted order IDs for geo/restaurant insight). All results pass through a **5-minute TTL cache**.

Endpoint	Returns
GET /calls?page=&page_size=	Paginated call list; each row enriched in Python with derived <code>agent_name</code> , <code>agent_helpfulness</code> , <code>customer_behavior</code> (keyword scoring of transcript turns)
GET /analytics/summary	5 queries: overview counts, intent distribution, weekly sentiment trend, top topics by frequency, top topics by negativity
GET /analytics/area-insights	Complaint/sentiment patterns by area (from transcript entities) + zones (from joined orders)
GET /analytics/restaurant-insights	Same, grouped by restaurant; adds ratings from <code>vendor_kpi</code>

8. Pillar 02 — Support Messages / Text Sentiment

Ingests customer text messages (App / WhatsApp / Ticket), classifies them with Gemini, and serves cross-channel analytics.

8.1 Ingest → classify → serve

```

INGEST  POST /api/v1/messages/upload (JSON) or /upload/csv
        _build_row(): uuid + preprocess() (PII redaction) + timestamps
        → bq_text.insert_messages() → reports.messages

CLASSIFY POST /api/v1/sentiment/classify {message_ids? , batch_size}
        • if no ids given → get_unclassified_message_ids() (LEFT JOIN where no classification)
        • batches → text_classifier.classify_message() → Gemini (JSON)
          {sentiment, sentiment_confidence, intent, intent_confidence, negative_trigger}
        • inserted into reports.classifications (model_version = gemini model id)
        • paced by classify_batch_delay_s between batches

SERVE   GET /api/v1/sentiment/results (classification ⋈ message, filterable + paged)
        GET /api/v1/messages , /messages/{id}
        GET /api/v1/sentiment/accuracy (classifications ⋈ labels)

```

8.2 Text analytics (text_analytics.py → text_analytics_service.py → bq_text.py)

Each endpoint accepts an optional `start / end` (YYYY-MM-DD) window, validated in the router, applied as a date predicate in SQL. The service aggregates raw BQ rows into typed Pydantic responses.

Endpoint	What it computes
<code>/analytics/sentiment-trend</code>	Weekly positive/neutral/negative counts + percentages (bucketed Monday-start)
<code>/analytics/top-negative-triggers</code>	Top-5 negative <code>negative_trigger</code> phrases; filters by merchant/zone/time-of-day; returns top zones/merchants + time-of-day distribution
<code>/analytics/cross-channel</code>	Compares Pillar 02 text vs Pillar 01 call volumes by intent (mapping call intents → text intent vocabulary), plus per-channel sentiment summaries
<code>/analytics/intent-distribution</code>	Intent counts + percentages
<code>/analytics/merchant-sentiment</code>	Sentiment split per merchant
<code>/analytics/zone-heatmap</code>	Negative-sentiment concentration by delivery zone

8.3 Accuracy evaluation

`/api/v1/sentiment/accuracy` joins `classifications` to human `labels` and computes overall + per-class sentiment/intent accuracy and a confusion matrix — the model-quality readout.

8.4 Chat-analytics (`chat_analytics.py`, prefix `/api/messages`)

A parallel read surface over the same text data plus `chat_history`, used by the chat-history views: `/stats`, `/list`, `/triggers`, `/sentiment-trend`, `/cross-channel`, and `/chat_id` detail.

9. Pillar 03 — Cancellation Prediction

Predicts the probability that a live order will be cancelled, explains why, and surfaces root-cause analytics. It has two prediction engines and degrades gracefully when the ML model has not been trained.

9.1 Feature engineering — the single source of truth

`app/utils/feature_engineering.py` is shared by *both* the offline training script and the online predictor, so training and serving can never drift. Key design rule: **no leakage** — only placement-time fields are used; every post-delivery field is listed in `LEAKAGE_FIELDS` and excluded.

- **Placement-time features:** hour/day/weekend, order value, distances, pre-order/new-customer/pro flags, payment type, device, platform, cuisine, zones.
- **Historical (rolling) features:** vendor 30-day cancel rate, zone 30-day cancel rate, customer prior-order count, vendor avg prep time — computed from *prior orders only* (rolling window, `closed="left"`) in training; looked up point-in-time from BigQuery at serving.
- A scikit-learn `ColumnTransformer` (median-impute + scale numerics; impute + one-hot categoricals) is saved inside the pipeline.

9.2 Training (scripts/train_cancellation_model.py — offline)

1. Pull Delivered + Cancelled orders from vendor_kpi
2. feature_engineering.transform(training=True) # leakage-free rolling features
3. Time-based split (recent 20% = test; 20% of remainder = val)
4. Handle imbalance: XGBoost scale_pos_weight vs SMOTE – keep the better
5. Train LogisticRegression (baseline), XGBoost (primary, Optuna-tuned for recall with a precision floor of 0.3), LightGBM (comparison)
6. Evaluate on held-out test → write artifacts/
cancellation_model.joblib, feature_pipeline.joblib,
model_metrics.json, threshold_analysis.json, threshold_recommendation.json,
feature_importance.json, pr_curve.json

Triggered offline or via `POST /api/cancellation/model/retrain` (which launches the script as a detached subprocess and returns immediately).

9.3 Serving (predictor_service.py)

Model + pipeline are loaded lazily from `artifacts/`. Engine selection (`engine` query param):

- `model` — ML only; raises 503 (`ModelUnavailable`) if untrained.
- `gemini` — LLM estimate from order signals; works with no trained model.
- `auto` — uses the model if trained, otherwise Gemini.

```
predict_one / predict_batch / live_queue / explain_order
├── model path: _fill_history() (BQ lookup) → pipeline.transform → predict_proba
│               → SHAP TreeExplainer top factors → (optional) Gemini explanation
├── gemini path: _fill_history() → _gemini_score_row() (LLM probability + factors)
└── ▼ bq_predictions.save_predictions() # persist as read-through cache
```

Read-through cache: `live_queue` first reads stored predictions for the candidate orders from `cancellation_predictions`; only *uncached* orders are scored (Gemini path capped at 20 new scorings/request so a cold cache can't hang the endpoint — the rest fill in across loads).

9.4 Cancellation analytics (cancellation_service.py)

Eight exploration queries over `vendor_kpi`, each windowable and cached for 5 minutes:

Endpoint (prefix <code>/api/cancellation/analytics</code>)	Breakdown
<code>/trend</code>	Monthly + weekly cancel rate
<code>/by-merchant</code>	Top merchants by volume and by rate (min 30 orders)
<code>/by-zone</code>	By <code>zone_name</code> and <code>customer_zone</code>
<code>/by-time</code>	By time-of-day bucket (Morning/Lunch/Afternoon/Dinner/Late Night)
<code>/by-day</code>	By day of week
<code>/by-order-size</code>	By order-value quartile
<code>/by-actor</code>	Who cancelled (<code>cancelled_by_txt</code>)
<code>/crosstabs</code>	Zone × time-of-day, merchant × day-of-week

The same query functions are reused by `scripts/explore_cancellations.py` to materialise the artifact JSONs in `artifacts/` — single source of truth, so endpoints and offline artifacts never diverge.

9.5 Gemini drivers report & model management

Endpoint	Behaviour
GET <code>/analytics/drivers-report?</code> <code>force=</code>	Structured Gemini report (executive summary, top drivers, high-risk segments, trend). Cached to disk; generated on first request or when forced.
GET <code>/analytics/feature-importance /</code> <code>/threshold-analysis</code>	Served from training artifacts (or <code>{available:false}</code> if untrained)
POST <code>/analytics/refresh</code>	Clears caches, re-runs exploration to artifacts, regenerates the drivers report
POST <code>/chat</code>	Free-text Q&A grounded on live cancellation exploration data (via <code>chat_service</code>)
POST <code>/explain/{order_id}</code>	On-demand prediction + natural-language explanation for one order
GET <code>/model/info</code> , <code>/model/health</code> , POST <code>/model/retrain</code>	Model metadata, readiness, and background retrain trigger

10. Conversational Assistant (POST `/chat`)

A Gemini-backed analytics chatbot (`chat.py` → `chat_service.py`). On each request it fetches every analytics source *in parallel* (call summary, area/restaurant insights, message stats, triggers, sentiment trend, cross-channel, vendor KPI) — each wrapped so a failure is non-fatal — and injects them as a JSON context block under a strict system prompt: the model may only cite facts present in the injected data, must answer in GitHub-flavoured Markdown, and must never invent SQL or statistics. Supports image attachments (multimodal).

11. Caching & Cost-Safety

Mechanism	Where	Effect
5-min TTL cache	call_analytics_service, cancellation_service	Collapses repeated dashboard loads (every mount + auto-refresh) to one BigQuery query per window; cleared on new writes / refresh
Read-through prediction cache	cancellation_predictions table	Avoids re-scoring (and re-billing Gemini for) the same order
Disk-cached drivers report	artifacts/	Slow Gemini report generated once, served from disk
maximum_bytes_billed (50 GiB)	bq_client default job config	BigQuery rejects any runaway full-table scan instead of silently billing it
Gemini semaphore (5)	gemini_service	Caps concurrent LLM calls

12. Offline Scripts & Pipelines

Script	Purpose
seed_text_messages.py	Generate synthetic support messages for Pillar 02
run_batch_classify.py	Bulk-classify unclassified messages (with <code>--dry-run</code>)
run_chat_pipeline.py	Process raw <code>chat_history</code> into classified messages
train_cancellation_model.py	Train the cancellation model + write artifacts (also Colab-runnable)
explore_cancellations.py	Materialise cancellation exploration JSONs (+ optional Gemini report)
backfill_message_sla.py	Add <code>closed_at</code> / <code>agent_name</code> to messages from <code>chat_history</code> (SLA basis)
backfill_message_enrichment.py	Backfill <code>customer_id</code> / <code>zone</code> / <code>merchant_name</code> from chat + order data

13. Complete Endpoint Reference

Method	Path	Description
GET	/health	Liveness
GET	/api/v1/health	BigQuery + Gemini reachability
POST	/analyse	Analyse one transcript (+ order enrichment + persist)
POST	/analyse/batch	Analyse ≤ 100 transcripts
POST	/predict	Quick intent+sentiment, no enrichment
GET	/calls	Paginated analysed calls
GET	/analytics/summary · /area-insights · /restaurant-insights	Pillar 01 aggregates
POST	/api/v1/messages/upload · /upload/csv	Ingest messages
GET	/api/v1/messages · /messages/{id}	List / fetch messages
POST	/api/v1/sentiment/classify	Classify messages via Gemini
GET	/api/v1/sentiment/results · /accuracy	Results + accuracy vs labels
GET	/api/v1/analytics/{sentiment-trend,top-negative-triggers,cross-channel,intent-distribution,merchant-sentiment,zone-heatmap}	Pillar 02 analytics (windowable)
GET	/api/messages/{stats,list,triggers,sentiment-trend,cross-channel,{id}}	Chat-history analytics
POST	/chat	Conversational analytics assistant
POST	/api/cancellation/predict · /predict/batch	Score order(s)
GET	/api/cancellation/predict/live-queue	Score active orders (cached)
GET	/api/cancellation/analytics/*	trend, by-merchant/zone/time/day/order-size/actor, crosstabs, drivers-report, feature-importance, threshold-analysis
POST	/api/cancellation/{analytics/refresh, chat, explain/{id}}	Refresh, Q&A, explain order
GET	/api/cancellation/model/{info,health}	Model metadata / readiness
POST	/api/cancellation/model/retrain	Background retrain